

Mortgage QA/QC Tool — Build Thesis

Synthesis of the strategy conversation in `docs/transcript.md` + `docs/summary.md` . What Gordon is building, why it matters, and the decisions that define it. Date: 2026-06-19

The One-Sentence Thesis

Build a configurable, deterministic QA/QC engine that lets a non-technical mortgage SME wire up 800+ closed-loan checks as routes → blocks → checks, run them on demand against three data sources, and produce a result set a human can clear in seconds — auto-clearing everything the machine can decide, and surfacing only the true exceptions that require human judgment.

This is not a one-off prototype. It is a **seed to be productized** — the POC caught the client's imagination, and the job now is to turn it into a real product.

The Core Problem (in the client's words)

A closed, funded loan file comes back from the title company — a blob of hundreds of signed, dated PDFs. That is the **source of truth**. Someone has to QC it against the loan data the lender holds. Today this is manual, slow, and judgment-heavy.

The tool's job: **apply the right checks to the right loan, correctly, every time — and give the reviewer a result set that auto-clears the obvious and isolates only what genuinely needs a human.**

"We auto-clear some, and the remainder are things that absolutely have to be decided by a human... Those are the only things they do on the platform. I'm done with this loan. Next one, next one, next one."

Point 1 — What to Build vs. What to Assume (Scope Discipline)

The single most important strategic decision in the meeting: **do not boil the ocean**. Attack the core of the problem, assume the periphery is solved. This came directly from the manager when Gordon described the two things "giving him fits" — both of which were *periphery*, not core.

Area	Decision	Why
Document data extraction	❌ Do not build. Hand to the Touchless team.	It was <i>assumed</i> to be a solved product already, but extraction inaccuracy was poisoning the QC. Rebuilding it is out of scope.
LOS integration	❌ Do not build. Reuse the existing connector.	"We have a way into the LOS today, we can reuse that."
The rules engine + config workbench + result set	✅ This is the product. Build this.	This is "what the tool is supposed to be really good at at the core."

"Let's not try to attack all the parts of the problem. Let's attack what this tool is supposed to be really good at at the core: I have 800 checks, I have three sources of data — can I apply them correctly and give a good result set? Solve for that first."

The handoff contracts (what "assume it's solved" actually means)

- **To Touchless (document analysis):** "I'm going to give you a document blob. Your job is to make sure it goes through the document analysis tool and gives me *this data back and the classification of the documents back*." The tool consumes that output; it does not produce it.
 - Caveat flagged: "*We may have to expand the extraction because we need more data elements to review.*" → the QC tool's data needs may push the extraction contract wider over time. Track that as an interface, not a build.
- **To the LOS:** an agent/connector already exists; the tool reads loan data through it.

Why this discipline matters

The failure mode is trying to solve extraction **and** integration **and** the rules engine simultaneously — and shipping none of them well. The core (apply 800 checks across 3 sources, return a clean result set) is the thing that earns the right to productize everything else.

"Rather than 'oh, let me solve the extraction problem, let me solve the integration problem' — no. Solve the core of the problem first."

Point 2 — Three Data Sources, Reconciled (Not Checked in Isolation)

The tool must ingest and **cross-compare** three independent representations of the same loan. The insight from the manager: checking one source against a rule isn't enough — the value is confirming the sources *agree*.

#	Source	Origin	Notes
1	Closed loan documents (PDFs)	Title company, post-closing	The source of truth. A funded, closed file signed & dated by the borrower at the table. Comes back as a blob of hundreds of PDFs. Touchless unpacks → classifies → extracts fields.
2	MISMO 3.4 XML	Title company or LOS export	The title company may send loan data in MISMO form alongside the documents; alternatively it can be produced from the LOS.
3	LOS export (3.4 file)	Loan Origination System	Pulled via the existing connector.

The two-step that is the actual product

"You have the document, and you extracted the data from it — and then you can match. It is two steps."

1. **Extract** the data from the document blob (done upstream by Touchless).
2. **Cross-compare** that extracted data against the LOS and MISMO data, then **apply the checks** — so a check can assert not just "is this value valid?" but "do all three sources tell the same story?"

Why this was a sticking point

Gordon flagged a trap: if all his test loans come *only* from the LOS, the document-vs-system comparison can't actually be tested — the data is trivially identical because it has one origin. The manager's correction:

"we need to consider all three." Real validation requires the document path and the system path to be genuinely independent. (This ties directly to the labeled-test-data blocker in Point 5.)

"All the data she's going to provide is from the LOS, so that will be totally accurate — I can't compare the document to the system." → "No, we need to consider all three."

Point 3 — The Defining Architectural Bet: Determinism Above All

This is what makes the product different from Olav's LLM-centric POC.

Non-negotiable requirement: the same loan must produce the same pass/fail **every time**. No "I thought about it and ran it this way, but next time differently."

"Even if it's an LLM, it has to be deterministic. We can't put an asterisk — 'LLM used, so there may be mistakes.' We can't do that."

The proposed design — "compile, then run"

Think of it as a **compiler stage** that sits between the human-authored rules and the runtime engine:

- The LLM works at **configuration time**, not run time.
- It interprets the rule spreadsheet / SME intent and **generates an intermediate rule set** (Gordon already prototyped exactly this: take the stated rules → convert to a Drools-style ruleset → run it through a plain rule engine afterward). His version lacked the citation/display polish but proved the mechanism.
- That generated ruleset is **validated, agreed to, and tested** by the SME *before* it ever runs in production.
- The engine then runs the compiled rules deterministically against every loan — the same compiled artifact every time.
- On update: take the change **through the pipeline** → regenerate the ruleset → re-validate → run. The LLM never freelances at runtime.

The hard requirement, stated three ways

1. **Reproducibility.** "Regardless of the loan that goes through, it won't give me a pass one time and a fail another." Same input → same output, always.
2. **No asterisks.** "We can't put an asterisk — 'LLM used, so there may be mistakes.' The AI is *used*, so it's assumed to validate itself." The tool is the thing that validates; it cannot offload uncertainty to the user.
3. **Even an LLM design must clear this bar.** The manager is explicit he's *not* dictating the implementation — "even if it's an LLM, it has to be deterministic." An LLM-at-runtime design is permitted **only if** it can prove it returns the right checks and the right result set every single time.

Two reasons this matters (the business case for determinism)

1. **Auditability / regulation.** A regulator sits on top and will audit *how* a number was calculated. "If they don't understand how you calculated that number, you are dead — you will buy back that loan." You must be able to show the exact equations applied. Determinism is a **compliance requirement**, not an engineering preference.
2. **Cost at scale.** Per-run LLM inference across thousands of loans could be punishing and is currently unknowable. "If it's a cent a file, nobody cares. If it's a couple dollars a file, everybody cares — we could have 10,000 files; every run costs me \$10,000." Compiling the ruleset once eliminates per-run token cost. **The PRD must estimate per-file token cost before committing to any runtime-LLM path.**

Open tension to resolve: Olav argues LLMs can repeat their behavior reliably enough to skip the intermediate ruleset and just run directly. The manager has heard him say it "a couple of times" but: **"I have no way of validating that."** → The burden of proof is on the runtime-LLM side; until proven, the intermediate compiled ruleset is the default. This is the central design debate the PRD exists to settle.

Empirical update — the G3 bake-off (2026-06-28)

We stopped asserting and measured it. A pre-registered head-to-head (decision rules locked *before* running; `p0/experiment_g3/`) put the compiled engine (Arm A) against a governed runtime-LLM (Arm B) at `temperature=0` , on the labeled golden loans, across **two models**. What the evidence actually said — including the parts that went *against* the thesis:

Axis	Engine A	Haiku 4.5	Sonnet 4.6
Determinism	bit-exact	byte-identical ×5	byte-identical ×5
Safety (false-auto-clear)	0	1 (cleared a 98%-LTV loan vs 95% max)	0 (caught it)
Cost / 10k-run (synthetic payload)	\$0.00	~\$27	~\$70

Two original arguments for compiling **did not survive on this data**:

- **"The LLM varies at runtime" is false here.** At `temp=0`, *both* models were byte-identical across 5 runs. Reproducibility alone does not distinguish the architectures.
- **The "\$10K/run" figure was wrong by 100–400× on these small synthetic payloads** — it came out at \$27–\$70.

Two arguments **strengthened**:

- **Runtime-LLM correctness is model-dependent and unknowable in advance.** Haiku *reproducibly* bought back a bad loan; Sonnet didn't. You cannot tell, before the fact, which you have — and you cannot hand a regulator the derivation either way. The compiled engine is correct *by construction* and shows the Decimal math + rounding policy. **This is now the load-bearing reason to compile**, not variance.
- **Cost still favors A decisively, for two reasons the synthetic test understates.** (1) The ~\$27–\$70 figure rides on tiny ~1.1K-token payloads; **real loans carry the full Touchless extraction (hundreds of fields, hundreds of pages)**, plausibly 10–50× the input tokens → Sonnet realistically **\$700–\$3,500/10k-run**, back in "everybody cares" territory (§Point 3.2). (2) It is **per run, and every rule change re-runs the whole portfolio**. Engine A is **\$0 at any payload size, any scale, any number of re-runs, forever**. For a tool sold to many lenders running QC continuously, that is a genuine moat, not a rounding error.

Resolution of the open tension: Olav's "just run the LLM" is *viable on the narrow axes he claims* (a strong model can be reproducible and, on clean data, accurate) — so we hold **runtime-LLM as a live option for the ambiguous, no-deterministic-algorithm cases** (the autonomy story, Point §Where This Fits). But for the deterministic core — applying 800 arithmetic/predicate checks at scale, under audit — **the compiled ruleset remains the default**, now justified on **auditability + guaranteed-correct math + zero marginal cost**, not on LLM flakiness. The decision is *not* settled by determinism alone; it is settled by audit and cost.

Still open (the real-data gate): accuracy here is *directional* — 6 hand-authored loans. The number that converts this from "compelling" to "proven" is the re-run on **Kayla's expert-labeled, independent-path loans**. If a strong model proves accurate enough on *real* loans, compiling becomes a governance/audit *preference* rather than a correctness *necessity* — but the cost argument for A stands regardless. Full writeup: `p0/experiment_g3/RESULTS.md` .

Point 4 — The Philosophy That Won the Room (Do Not Stray From It)

This is the part that's already validated by the market — the client reacted to it viscerally, and the manager's instruction is to **protect it**, not reinvent it. "We have a nice seed. Let's turn this into a product — and don't stray from the philosophy of the tool."

The configuration model: Routes → Blocks → Checks

- **Routes** are composed of **blocks**; blocks are wired together by hand; **checks** live inside blocks.
- Point the configured route at a target set and **run on demand, at a whim**.
- The same primitives scale both ways: "I can configure a very simple thing, or configure a very complex thing — it's up to me."

Who configures it — and why that's the whole point

The buyer is **non-technical**: the BA or subject-matter expert builds and runs this **without going back to IT**. That self-service capability is what caught the client's imagination — "I could see it in their face" — and David reiterated it in this very meeting:

"He wants his non-technical folks to be able to configure this easily. We heard that loud and clear."

The reaction the product is engineered to reproduce: *"Wow, I can use this tool to configure something simple or something very complex. The BA or SME can build this out and run files against it — I don't need to go back to my IT."*

Design mandate — the three surfaces to perfect

The manager named these explicitly as where the effort goes (and where it does **not** — not extraction, not integration):

1. **Apply** — how rules are applied (the deterministic engine of Point 3).
2. **Author** — a configurable pattern that lets non-technical users *create* the rules.
3. **Output** — present the result set so a human can mitigate an outcome **very quickly**. Auto-clear what the machine can decide; surface only the human-judgment exceptions, fast.

Point 5 — Known Build Blockers (Raised by Gordon, With Mitigations)

These are the three things Gordon said were "giving him fits." Each has a sanctioned mitigation from the meeting.

Blocker 1 — Extraction accuracy (input quality)

Bad input poisons the QC: "I had to have accurate information to be able to QC, and that was the part having trouble." Extraction was assumed to be a solved product but wasn't reliable enough.

- **Mitigation:** Don't rebuild it. Lean on Touchless, and in the near term use **Kayla's files already processed through Cloud** — which may remove the need for Gordon to do document extraction at

all for the prototype. Treat extraction as an upstream contract (see Point 1).

Blocker 2 — No labeled test data (the eval gap)

Gordon gets raw files but doesn't know what's wrong with them: "If it gives me an error, I don't know if it's right or wrong." Without known outcomes he can't tell whether a fired rule is *correct*.

- **What's needed:** synthetic or pre-validated loans with **known, expert-checked outcomes** — "credit score is not enough," "the appraisal is off" — so that when those rules fire, he *knows* the engine is working.
- **Mitigation:** Kayla provides a labeled subset she has personally validated ("I understand exactly what needs to happen, how the check should be implemented, and this is the result that should come out"). She must also **validate the check interpretations themselves** — did the team read each of the 800 checks correctly?
- **Why it's foundational:** This is an **eval problem**. The tool is only as trustworthy as the ground truth it's measured against — and it connects to Point 2: test data must include genuinely independent document and system sources, not LOS-only data, or the cross-comparison can't be validated.

Blocker 3 — Rule-to-program mapping unknown (scope of the 800)

The client's Excel spreadsheet is the **base** rule set, but it doesn't say *which* rules fire for *which* product/program. "We don't want to run all 800 for every loan." Owner-occupied vs. investment loans apply **distinct** rule sets, not similar ones.

- **Open questions:** Is the spreadsheet *all* we need, or do we also pull from the Fannie/Freddie selling guides? ("That's our base, but we may need additional information to implement it.")
- **Mitigation:** Kayla to get deeper interpretation from the client on rule applicability. **For now, assume the rules apply to everything; structure the product/program gating later.** Don't block the build waiting on this.

Immediate Next Step: Write the PRD

The deliverable that turns the seed into a product. Structure it as input → tool → output, asking every open question:

- **Input:** the three sources; what extraction must return; what data elements need expanding.
- **Tool processing & design:** LLM vs. non-LLM; intermediate ruleset vs. not; the determinism guarantee; cost model.
- **Configuration:** the routes/blocks/checks authoring pattern for non-technical users.
- **Output:** result-set format optimized for fast human mitigation; auto-clear logic.
- **Tentacles / integrations:** LOS connector, Touchless document analysis, where results are sent.

Owners: Gordon + the team of three + Kayla. **Timeline:** ~1 month. **Then:** hand the validated prototype to Monish's team for industrial-strength build-out (observability, monitoring, auditability, security, guardrails) on the Touchless platform.

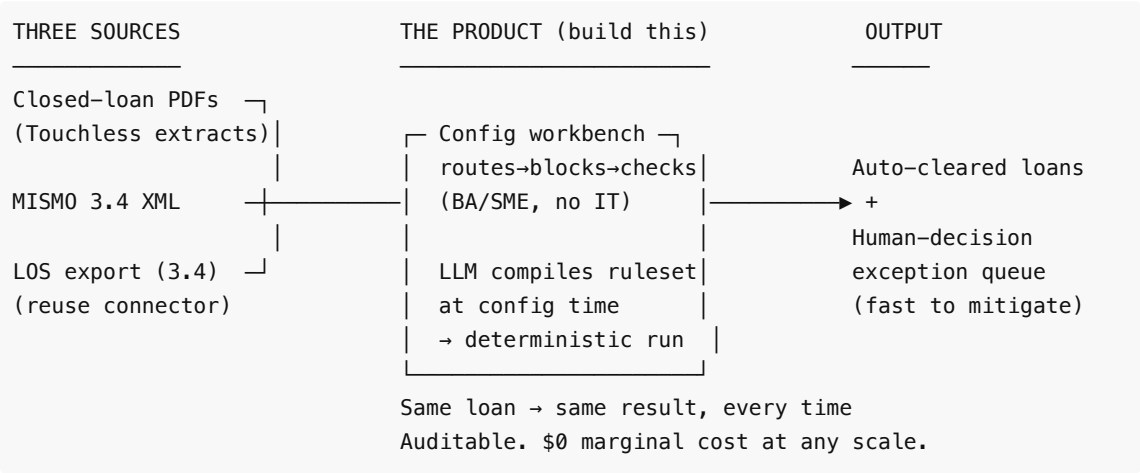
Where This Fits the Bigger Picture

- **The operating model:** Gordon prototypes fast → validates with Kayla/client → hands off to Monish for production. This QA/QC tool is the canonical example of that flywheel. The same pattern applies to a title-analysis component and to ideas thrown out by the Citizens prospect ("if we could prototype that quickly, he'll go with us — nobody in the industry is doing it").

- **HousingWire AI Summit:** Olav handles it. Format that resonates = **theory/principles → live tool walkthrough → proof**. The QA/QC tool is itself a textbook example: "here's the 800 things they gave us, here's the process, here's the tool that lets you configure rules — and look, it works."
- **Mortgage AI Conference (California, Oct), Gordon's keynote target:** Build something **truly autonomous** over July–September — an agent that, within guardrails, figures out the fix the way a human would, in a case where no deterministic algorithm exists. The QA/QC tool is the determinism story; the October keynote is the *autonomy* story. (Enabled by Fable.) Brainstorm with Sandeep, Olav, others.

The nuance Gordon must internalize: agentic AI belongs **only** where there's no deterministic algorithm (e.g., an ambiguous income case an underwriter must reason through). Where an algorithm exists — most income analysis — you want "complete, utter determinism" because the regulator will audit the math.

The Mental Model (TL;DR)



Build the engine and the workbench. Assume extraction and LOS are solved. Make it deterministic. Keep it configurable by non-technical users. Write the PRD first.